

Computer Graphics Project Proposal

Department of Computer Science & Engineering
American International University Bangladesh (AIUB)

1. Title

Iran–USA Border Conflict Zone: A 2D Computer Graphics Scene

2. Team Member List

Name	Student ID	Contribution
Suaib Ahmmed Nafees	23-50723-1	Introduction
Salman Zzoha	22-49602-3	Description of the Environment and Methods
Sadia Afrin	23-54513-3	Feature set
Samiha Sultana	23-54895-3	Conclusion

3. Introduction

This project features a computer graphics simulation called "Iran-USA Border Conflict Zone." It illustrates a fictional military standoff at an international border during the night. Using basic 2D graphics programming techniques, the scene shows two opposing forces: the United States military on the left and the Iranian military on the right. They are separated by a heavily fortified border wall, fence, and barriers. The scene includes military vehicles like tanks and fighter jets, watchtowers with searchlight beams, soldiers, national flags, and a central explosion with rising smoke. This combination creates a dramatic and visually striking environment.

The main reason for selecting this scenario is its variety of shapes and complexity. A military border scene naturally requires various graphical elements, such as rectangles for buildings and vehicles, circles for wheels and explosions, lines for fences, barbed wire, and rifle barrels, polygons for jets and missile trails, and ellipses for clouds and smoke. Each of these shapes connects directly to basic functions in the graphics.h library, making this an excellent opportunity to practice essential computer graphics concepts.

Projects like this have been popular in academic settings to teach students how to build complex environments from simple shapes. Other similar scene-based projects include village simulations, solar system animations, and traffic systems, each showing how real-world settings can be recreated using basic drawing methods. Our project builds on this tradition by creating a more dramatic environment that tells a story, highlighting the differences between the two sides through color palette, architectural style, and military equipment design.

The project aims to fully implement a static 2D nighttime military border scene, displayed in a 1024x480 pixel window. The scene is split into four zones, each corresponding to one of the team members, who handle specific areas and graphical elements. Optional animation features, such as a rotating helicopter rotor, blinking explosions, or moving missiles, could be added in future versions using the `delay()` and `cleardevice()` functions. This project offers a thorough exercise in coordinate planning, color management, shape composition, and collaborative software development.

4. Description of the Environment and Methods

The development of the project is carried out using the C++ language with the Code::Blocks IDE in the Windows 10/11 OS. The output is performed with the help of functions from the OpenGL library, using the GLUT (OpenGL Utility Toolkit), which provides an interface for rendering graphics and handling window operations in Windows systems. This library allows performing two-dimensional graphic operations efficiently, which is convenient for educational projects.

The development environment configuration requires linking the OpenGL libraries (`opengl32`, `glu32`, `glut32`) in Code::Blocks and adding the necessary header files such as `<GL/glut.h>`. The creation of a graphical interface is provided using GLUT functions like `glutInit()` and `glutCreateWindow()`, which open a graphical window with a resolution of 1024x480 pixels. Such a wide format of a display window was chosen to create a scene with a border zone landscape, where two military parties are located opposite each other.

Among the main graphics functions that are implemented in the program, there are functions for creating geometric shapes using primitives such as `glBegin()` and `glEnd()`, along with vertices defined by `glVertex()`. Different shapes such as lines, triangles, quadrilaterals, and polygons are drawn using modes like `GL_LINES`, `GL_TRIANGLES`, `GL_QUADS`, and `GL_POLYGON`. In addition, functions for setting colors (`glColor3f()`), clearing the screen (`glClear()`), and defining background color (`glClearColor()`) are used

in the project. The rendering process is controlled through the display() callback function, which continuously updates the graphical scene.

The night sky color scheme is achieved by using dark navy and charcoal background colors, with warm yellows and oranges for explosion and spotlight effects.

5. Feature Set

Every graphics function represents a particular drawing operation or act that needs to be performed in the scene.

Member 1 — Sky, Atmosphere & Explosion (20 Features)	
1.	Draw the night sky background using a dark navy rectangle covering the full window
2.	Draw the moon using a filled white/cream circle in the upper right area
3.	Draw the moon's shadowed crescent using a dark circle overlapping the moon
4.	Draw at least 12 scattered stars using small filled circles with varying sizes
5.	Draw Cloud 1 using overlapping ellipses on the left portion of the sky
6.	Draw Cloud 2 using overlapping ellipses in the center-upper portion
7.	Draw Cloud 3 using overlapping ellipses on the right portion of the sky
8.	Draw a large explosion burst using a filled orange-red circle at the border center
9.	Draw the explosion inner glow using a smaller yellow-white filled circle
10.	Draw rising smoke cloud 1 using a dark gray ellipse above the explosion
11.	Draw rising smoke cloud 2 using a medium dark ellipse slightly offset from cloud 1
12.	Draw rising smoke cloud 3 using a smaller ellipse at the top of the smoke column
13.	Draw flying debris particle 1 using a small filled polygon near the explosion
14.	Draw flying debris particle 2 using another small shape on the opposite side
15.	Draw a missile trail line using a dashed or dotted diagonal line from right to center
16.	Draw a missile tip using a small filled triangle at the end of the trail
17.	Draw the spotlight beam from the USA watchtower using a filled transparent polygon
18.	Draw the spotlight beam from the Iran watchtower using a filled transparent polygon
19.	Draw a fire glow effect at the explosion base using a wide flat orange ellipse
20.	Draw scattered spark lines (at least 3) radiating outward from the explosion center

Member 2 — Border Zone, Fence & Ground (20 Features)	
1.	Draw the USA ground terrain on the left half using a filled sandy/brown rectangle
2.	Draw the Iran ground terrain on the right half using a slightly different shade rectangle
3.	Draw the border wall center column using a tall dark gray rectangle
4.	Draw border fence wire row 1 using a horizontal line across the wall

5.	Draw border fence wire row 2 using a second horizontal line below the first
6.	Draw border fence wire row 3 using a third horizontal line completing the fence pattern
7.	Draw the left border pillar using a vertical filled rectangle on the left edge
8.	Draw the right border pillar using a vertical filled rectangle on the right edge
9.	Draw the border signboard rectangle using a small colored rectangle at the top of the wall
10.	Draw the border label text 'BD USA IRN' using outtextxy() on the signboard
11.	Draw the barbed wire on the USA side using a jagged polyline along the ground level
12.	Draw the barbed wire on the Iran side using a matching jagged polyline
13.	Draw sandbag stack 1 using stacked ellipses on the USA side near the border
14.	Draw sandbag stack 2 using stacked ellipses on the Iran side near the border
15.	Draw a bunker entrance arch on the USA side using a semicircle shape
16.	Draw a trench line on the USA ground using a narrow dark rectangle
17.	Draw a trench line on the Iran ground using a matching narrow rectangle
18.	Draw a checkpoint gate bar using a horizontal line across the road at the border
19.	Draw gate support post 1 using a vertical rectangle on the left
20.	Draw gate support post 2 using a vertical rectangle on the right

Member 3 — USA Side: Watchtower, Tank, Soldier & Jet (20 Features)	
1.	Draw the USA watchtower base body using a tall filled olive-green rectangle
2.	Draw the USA watchtower observation roof using a wider rectangle on top of the body
3.	Draw the USA watchtower support leg using a narrow rectangle below the main body
4.	Draw watchtower window 1 using a small light blue filled rectangle on the left side
5.	Draw watchtower window 2 using a matching small rectangle on the right side
6.	Draw the USA flag pole using a thin vertical line above the tower
7.	Draw the USA flag red stripes using alternating red and white horizontal rectangles
8.	Draw the USA flag blue canton using a small blue rectangle in the top-left of the flag
9.	Draw the USA tank hull/body using a wide dark green filled rectangle
10.	Draw the USA tank turret using a smaller rectangle centered on top of the hull
11.	Draw the USA tank gun barrel using a long thin horizontal line extending from the turret
12.	Draw USA tank wheel 1 using a filled dark circle under the left side of the hull
13.	Draw USA tank wheel 2 using a filled dark circle under the center of the hull
14.	Draw USA tank wheel 3 using a filled dark circle under the right side of the hull
15.	Draw the USA tank tread using a thin rectangle connecting all three wheels
16.	Draw USA soldier head using a small filled skin-tone circle
17.	Draw USA soldier body using a small olive-green rectangle below the head
18.	Draw USA soldier left arm using a diagonal line from the body

19.	Draw USA soldier right arm holding rifle using a line with a small rectangle (gun)
20.	Draw USA fighter jet body using a filled gray polygon/triangle shape in the upper left sky

Member 4 — Iran Side: Watchtower, Tank, Soldier & Jet (20 Features)	
1.	Draw the Iran watchtower base body using a tall filled brown/tan rectangle
2.	Draw the Iran watchtower observation roof using a wider rectangle on top of the body
3.	Draw the Iran watchtower support leg using a narrow rectangle below the main body
4.	Draw watchtower window 1 using a small light blue filled rectangle on the left side
5.	Draw watchtower window 2 using a matching small rectangle on the right side
6.	Draw the Iran flag pole using a thin vertical line above the tower
7.	Draw the Iran flag green stripe using a top horizontal filled rectangle
8.	Draw the Iran flag white stripe using a middle horizontal filled rectangle
9.	Draw the Iran flag red stripe using a bottom horizontal filled rectangle
10.	Draw the Iran flag emblem (simplified) using a small shape in the center of the flag
11.	Draw the Iran tank hull/body using a wide filled tan/desert-color rectangle
12.	Draw the Iran tank turret using a smaller rectangle centered on top of the hull
13.	Draw the Iran tank gun barrel using a long thin horizontal line extending from the turret leftward
14.	Draw Iran tank wheel 1 using a filled dark circle under the left side of the hull
15.	Draw Iran tank wheel 2 using a filled dark circle under the center of the hull
16.	Draw Iran tank wheel 3 using a filled dark circle under the right side of the hull
17.	Draw the Iran tank tread using a thin rectangle connecting all three wheels
18.	Draw Iran soldier head using a small, filled skin-tone circle
19.	Draw Iran soldier body using a small brown military-color rectangle below the head
20.	Draw Iran fighter jet body using a filled tan/brown polygon shape in the upper right sky area

6. Conclusion

This project is about making a 2D computer graphics scene that looks like a war zone between the USA and Iran at night. The picture will be nice to look at and not too hard to make using basic graphics tools in Code::Blocks. We are dividing the scene into four parts. The sky, the border, the USA side and the Iran side. This way each of the four team members can work on a part. Make 20 different things for it.

The project shows how to use computer graphics ideas like drawing with coordinates filling colors making shapes and creating complex scenes from simple shapes. We picked this scene because it tells a story and has different shapes so each team member can try out many different drawing tools and still take a picture that looks good.

There are some limits to this project. Now it is just a still picture and does not move or let people interact with it. All the things in the picture are drawn by hand using fixed coordinates, so we must plan so things do not overlap. Maybe later we can add movement to the picture using codes to make things like tanks, helicopter blades or missiles move.

We could also add ways for people to interact with the picture using the keyboard or mouse or change from day to night or add sounds. These things are not part of the project now. They could be added later to make it better.